

On-device Visual Perception for Light-weight Browser Agents

SIH 26171 — Prototype Report
Smart India Hackathon 2026 · Research track

- Problem statement**

Build a privacy-preserving browser agent that understands a web page *on the user’s own device*, redacts all personal data *before* anything leaves the machine, and only then asks a local language model to decide the next action — enabling a user to automate browsing tasks (form-filling, searches, navigation, reading dashboards) without ever shipping screenshots full of Aadhaar numbers, PAN cards, bank details, or faces to a remote server.
- Approach in one line**

Sanitise on-device (→ the VLM never sees raw PII), ground every action by element ID, decide with a local Qwen2.5-VL, adapt with a structurally-vetoing recovery planner.

1 Scoring Criteria

The judges weight five criteria. Every number in this report maps to one of them:

Criterion	Weight	What judges look for
Visual-context accuracy	25%	Can the agent complete real browsing tasks? Does the VLM correctly interpret the page and choose the right action?
PII detection recall / precision	20%	Does the system catch all personal data (recall) without flagging non-PII (precision)? Measured pixel-level on ground-truth-labelled test pages.
Redaction precision	20%	Of the pixels the system redacted, how many were actually PII? High precision = minimal over-redaction = the VLM still sees enough context to act.
Client resource utilization	20%	How heavy is the on-device work per step? Model size, bundled assets, vision latency, OCR-gate cost — all measured, all local.
End-to-end latency	15%	How fast is one full step (capture → sanitise → VLM → execute)? Target: < 5 s/step on a judge-class GPU.

How the metrics are measured

All accuracy metrics are computed on **rasterised redaction masks** — the union of predicted redaction bounding boxes versus the union of ground-truth boxes, clipped to the captured

viewport. This is a set-based measure (not per-element), so recall and precision are bounded by 1 by construction. Ground truth is encoded as **data-gt** attributes on every PII element in the test pages (including AI-generated faces and PII drawn into **canvas** elements). An automated Playwright harness loads each page, runs the sanitiser, computes pixel overlap, and writes results to JSON — fully reproducible, zero manual scoring.

2 How the Prototype Works

The prototype is an **end-to-end autonomous browser agent that runs a closed loop on the user's own machine**. A screenshot and a structured DOM snapshot are captured, **sanitised on-device so no personal data ever leaves**, and only the redacted payload is sent to a **local** vision-language model (VLM), which returns one JSON action to execute back in the tab. The loop repeats until the task is done.



Figure 1: End-to-end closed-loop architecture. Only the redacted image and numbered DOM ever leave the extension; only a JSON action ever comes back.

2.1 The components

Chrome Extension (WXT framework, TypeScript, Chrome MV3) — four entrypoints

Entrypoint	Role
background.ts	Orchestrator. Drives the loop: <code>captureVisibleTab</code> → <code>sanitise</code> → <code>zero-leak gate</code> → <code>POST /act</code> → <code>execute</code> in tab. Handles throttled step scheduling, stuck detection, and recovery routing to <code>/rethink</code> .
content-scripts/	Executor + DOM serializer. The executor converts JSON actions into real browser events (synthetic pointer/keyboard with a visible virtual cursor). The DOM walker serialises the page into a compact, numbered snapshot (shadow-root aware, capped at 120 elements).
sidepanel/main.ts	Control UI. Task input, “Run one step”, capture preview, live activity log, and a live “thinking” readout of the VLM’s tokens via SSE streaming.
offscreen/main.ts	On-device vision host (offscreen document). MediaPipe BlazeFace face detection and Tesseract OCR run here, in a DOM context, with all weights bundled locally.

Server, VLM, test site, benchmarks

- **Server** (FastAPI, Python) — a thin app over a local Ollama instance via the OpenAI-compatible API. Endpoints: `/health`, `/warm`, `/act`, `/act/stream`, `/rethink`, `/rethink/stream`, `/models`. It builds the VLM prompt (image + DOM), calls the model, and returns a validated action response. It also mounts the test site as static files.
- **VLM** — **Qwen2.5-VL 3b / 7b** running locally in Ollama. Ollama pins the model resident (`keep_alive:-1`), so the 20–60 s cold-load cost is paid once, not every step.
- **Test site** — seven ground-truth-labelled HTML pages (flight booking, bank transfer, faces, PII-in-the-wild, PII-in-canvas, and more). Every PII element carries a `data-gt` attribute describing its true category and bounding box. This is the source of all accuracy numbers.
- **Benchmarks** — an automated Playwright harness + Node.js runners that load each test page, run the sanitiser, compute pixel-level detection/redaction metrics, run the full VLM agent loop, and merge everything into a single scoring-aligned dashboard.

3 Tech Stack

Layer	Technology	Why this choice
Extension framework	WXT + TypeScript, Chrome MV3	Modern, type-safe extension build; shared modules between client and server; single build produces an unpacked <code>.output/chrome-mv3</code> directory ready to load.
Capture / serialise	<code>chrome.tabs.captureVisibleTab</code> + shadow-root-aware DOM walker	Real pixels (not devtools-protocol) plus precise bounding boxes (CSS <code>rects</code> × <code>devicePixelRatio</code>) for grounded VLM decisions.
On-device vision	MediaPipe Tasks Vision (BlazeFace short-range) + Tesseract.js WASM (English)	Face blur + image OCR entirely on-machine, zero cloud calls, ~2.2 MB of bundled model weights.
Privacy gate	Core sanitizer modules (main sanitizer, PII detection rules, zero-leak scanner)	Single choke point through which all outbound data must pass; no other code path can build an outgoing request.
Action protocol	Frozen v1 JSON protocol — TypeScript types on client, Pydantic models on server	One schema shared by both sides; a type mismatch is a compile/build error, not a runtime surprise.
Server	FastAPI (Python) with SSE streaming	Typed request/response models; Server-Sent Events for a live “thinking” readout; static-file mount for the test site.
VLM	Qwen2.5-VL 3b / 7b via Ollama (OpenAI-compatible API)	Local vision-language model; <code>keep_alive</code> pins it resident; dynamic 3b/7b routing trades accuracy for speed per step.
GPU tuning	Flash-attention + Q4_0 KV cache + single-resident model	Fits a vision model on a 4 GB laptop GPU (RTX 3050).
Test dataset	Ground-truth-labelled HTML pages with <code>data-gt</code> attributes	Enables fully automated, reproducible accuracy measurement — no manual annotation per run.
Benchmarks	Playwright harness + Node.js metric runners	Detection P/R, redaction precision, zero-leak checks, per-step latency waterfall, full VLM agent loop — all automated.

Deployment shape

Everything runs on the user’s machine by default. The extension is loaded unpacked from the build output, and the FastAPI + Ollama pair bind to `127.0.0.1`. There is **no external-service dependency** for the core loop. The VLM is the only component that could theoretically be swapped for a cloud API, but the default and demonstrated configuration is fully local.

4 Technical Approach (Deep Dive)

4.1 Privacy-first design: “sanitise before you send”

The central design decision is that **the browser is the trust boundary**. Every outbound request is built by the sanitizer module — a single choke point. The server receives only the

redacted screenshot and a numbered DOM. The server never sees raw pixels, and in the local setup never talks to the internet.

This is **defense in depth**, not a single heuristic:

1. Regex redaction of detectable PII (Tier A/B, §4.2).
2. Deterministic field-label flagging for sensitive inputs (flags `card`, `aadhaar`, `otp`, `password`, etc. even when the field is empty).
3. Vision-based redaction of faces and image-internal PII (Tier C).
4. A zero-leak re-scan of the **outbound JSON** (DOM regex).
5. A zero-leak re-scan of the **sanitised image pixels** (OCR of what would actually be uploaded).

A failure at any layer blocks the upload. There is no “soft fail” path that sends data anyway.

4.2 Three-tier redaction

Tier A — passwords / secrets / hidden fields

Solid black box, no text, no token. These are never useful to the VLM and always sensitive. Detected by exact field-name match (`password`, `passwd`, `pwd`, `cpassword`, `confirmpassword`, `retypepassword`, `secret`, `currentpassword`, `newpassword`, `cvv`, `otp`, `pin`) and by substring match (`api_key`, `api-key`, `secret`, `token`, `cvv`, `otp`). Also applied by element type to `<input type="password">` and `<input type="hidden">` (the DOM serializer returns `[SECRET]` for these).

Tier B — regex-detectable PII

Black box + a drawn token. **Same raw value always maps to the same token** across steps, so the VLM keeps semantic continuity (it can reason about `[EMAIL_1]` the same way across a multi-step form) without ever seeing the actual email.

PII type	Detection method	Token format
Email	Standard email regex	<code>[EMAIL_N]</code>
Phone (10-digit, optional +91)	Indian phone format	<code>[PHONE_N]</code>
Aadhaar (12-digit, first digit 2–9)	Indian Aadhaar format	<code>[AADHAAR_N]</code>
PAN (5 letters, 4 digits, 1 letter)	Indian PAN format	<code>[PAN_N]</code>
Credit card (13–16 digits)	Grouped digits + Luhn check	<code>[CARD_N]</code>
Name / address / DOB / IFSC / account	Field-label heuristics	<code>[NAME_N]</code> , <code>[ADDRESS_N]</code> , ...

The Luhn validation on credit cards is real (not just a digit count) — it prevents false positives on 16-character digit sequences that are not valid card numbers.

Tier C — faces and image-internal PII

- **Faces:** MediaPipe BlazeFace (short-range model, ~0.23 MB) detects faces in the captured screenshot; detected faces are “crushed-blurred” (downscaled to a few pixels, then upscaled) — an irreversible one-way transform that destroys facial identity.

- **Image-internal PII:** Tesseract OCR runs only on `img` / `canvas` / `video` regions (never on DOM text, already handled by Tier B). This catches PII that lives *inside pictures* — a scanned certificate, a photographed bank card, a screenshot of a form. **Taint policy:** once any PII is found in a region, the *entire region* is redacted, because pixel-level precision inside a raster image is unreliable.
- **Bank-specific:** account numbers and IFSC codes also get label-based rules (e.g. fields named “account number”, “IFSC” are flagged even if the regex does not fire).

Account numbers are OCR-only by design: in DOM prose, “dashed-alnum” tokens like EB-CA98765 are too common (order IDs, reference numbers) to regex without massive over-redaction. Running this rule only on OCR output — where there is no label context to lean on — is the deliberate trade-off.

4.3 Zero-leak guarantee

Two independent, **fail-closed** checks run before anything uploads:

Gate 1 — DOM regex scan (`scanForLeaks`). Iterates every element in the outbound DOM JSON and runs the full `detectPii` regex suite (email, phone, Aadhaar, PAN, Luhn-validated card, plus OCR-only account rules) over each element’s `text` and `value`. Any hit blocks the upload and logs which element leaked what.

Gate 2 — sanitised-image OCR (`scanSanitizedImage`). OCRs the *sanitised screenshot pixels* — i.e. what would actually be uploaded — and reports any surviving PII hits. This is the pixel-level check that catches what the DOM regex cannot see (text baked into images, partial redactions, rendering artifacts).

Fail-closed by construction. If Gate 2 cannot run (exception, timeout, DOM context unavailable) it returns `pass:false`. A gate that cannot run must not silently approve. Both gates must explicitly pass for the step to proceed. This is what allows the honest claim of **zero-leak PASS** on both the DOM and image channels.

4.4 Grounding by element ID, not coordinates

The DOM walker assigns stable integer IDs to elements; the Set-of-Marks overlay draws those numbers ([7], [12], ...) on the sanitised screenshot. The VLM returns `{"type":"click","target":7}`; the executor resolves ID 7 to the live DOM element, scrolls it into view, and dispatches events.

Server-side target validation (`resolve_target`). The server coerces and validates every target before returning it to the extension. It accepts plain integers (7), bracketed strings (“[7]”), numeric strings (“7”), and loose word matches against the DOM (ranked: a textbox beats a button beats a div). If the target cannot be mapped, the server returns an error and retries with a “fix hint” that reminds the model of the correct format.

Executor hardening.

- **Auto-descend:** if the VLM targets a non-typeable wrapper (a `<div>`, a `<yt-searchbox>`, a `<button>` containing an `<input>`), the executor automatically finds and types into the real inner `<input>`, records it as a verified target, and injects it into the next prompt as “*use VERBATIM, do not re-guess*” — so the model stops re-targeting wrappers.
- **Clicked-typeable memory:** the executor knows deterministically whether a click focused a real text field; if the VLM clicks the same input a second time without typing, the loop catches

- this “forgot to type” failure on the second repeat and steers the model toward typing.
- **Virtual cursor:** a visible cursor flies to the target and clicks, giving visual feedback and dispatching proper pointer events (not just DOM-level `.click()`) — important for sites that listen for `pointerdown/pointerup`.

4.5 Adaptive loop with recovery memory

- **Per-step history:** the last K steps are sent to the VLM on every call, including each step’s action, result, stated sub-goal, and whether it was blocked. This keeps the model on track across a long task.
- **Within-run warnings:** actions already tried that failed (or repeated without progress) are injected as a *DO-NOT-REPEAT* list — the model is told it already tried them and they did not work.
- **Cross-run lessons:** verified per-host lessons persist between runs (e.g. “on this site, the search box is inside a shadow root — target the outer wrapper first”) and are re-injected on the next run for that site.

Stuck detection → **/rethink**. Three signals trigger the recovery path: (1) the model repeated the exact same action twice; (2) no progress on the DOM fingerprint; (3) the model returned `blocked:true`. The **/rethink** endpoint sends the full history + sanitised screenshot with a forced alternative-plan prompt. Crucially, it **structurally vetoes** repeats: every action signature in history is checked, and an exact match is rejected with a “try something genuinely different” hint. If the model insists twice, a **deterministic escape hatch** clicks the first untried labelled element; if every element has been tried, it presses Escape. **/rethink** also refuses to return `done` just to escape the loop when the task is not actually complete.

4.6 Latency engineering

The bottleneck is VLM inference (CPU-bound on a 4 GB GPU where the 7b model runs 75% CPU / 25% GPU). Several levers reduce per-step cost:

Lever	Effect
VLM_MAX_TOKENS=128 (was 256)	Halves maximum decode length; the model’s actions are short JSON, so 128 tokens is ample.
SYSTEM_PROMPT_MODE=compact	~60% fewer system tokens; identical behaviour, shorter context.
IMAGE_MAX_SIDE=800 (Pillow resize)	Caps the longest JPEG dimension before upload, reducing tokens in the vision encoder.
keep_alive=-1 (Ollama)	Pins the model in VRAM between steps; avoids the 20–60 s cold-load penalty.
Dynamic 3b/7b routing	Interactive steps (click/type/search) use fast <code>qwen2.5v1:3b</code> (~7.5–8.0 s/step); read-and-report steps use accurate <code>qwen2.5v1:7b</code> . Net: 3/3 tasks at ~12.7 s/step avg (32% faster than 7b-alone) while holding 100% accuracy ; interactive steps cut 54%.
On-device vision budget	Vision + zero-leak OCR gate bounded to ~0.9 s total per step.

Hardware reality. On the dev laptop (RTX 3050, 4 GB VRAM), `qwen2.5v1:7b` (6.2 GB model) runs mostly CPU-bound. Vision tokens floor at ~1024 per image regardless of resolution, so prefill (~5.5 s) + decode (~5.5 s) are CPU-limited. The sub-5 s/step target requires VRAM ≥ model size (judge-class hardware) or a GPU-

resident small model. The routed 3b/7b design is what makes the agent usable on memory-constrained machines today.

5 Measured Results

All numbers below come from the automated benchmark harness and are fully reproducible.

5.1 PII detection and redaction

Metric	Result
PII detection F1 (pixel)	0.979
PII detection precision (pixel)	0.959
PII detection recall (pixel)	1.000
Redaction precision (pixel)	0.959
Zero-leak — DOM regex	PASS
Zero-leak — sanitised-image OCR	PASS
Face detection (Tier C)	2/2 faces blurred, F1 1.0
Canvas PII via OCR — account numbers	2/2 detected (canvas + input regions), F1 1.0
Canvas PII via OCR — IFSC codes	1/1 detected, F1 1.0

The 1.000 recall means the system caught *every* ground-truth PII element across all test pages. The 0.959 precision means about 4% of redacted pixels were not PII — mostly over-redaction in image regions where the taint policy redacts the whole bounding box once any PII is found inside it (deliberate trade-off: over-redact rather than leak).

5.2 Visual-context accuracy (VLM task completion)

Three ground-truth tasks with a 12-step budget each:

Task	Description
flight-booking	Navigate to a flight search page, fill origin/destination/date, submit.
bank-transfer	Navigate to a bank login, enter credentials, reach the transfer page.
pii-in-the-wild	Read a page containing mixed PII and report specific information.

Configuration	Tasks passed	Avg steps/task
qwen2.5v1:7b	3/3 (100%)	1.3
qwen2.5v1:3b	2/3 (67%) — fails prose “read & report” task	—
Routed 3b/7b	3/3 (100%)	1.0

The 3b-vs-7b gap on the reading task is *intended*: it proves the harness discriminates model capability — exactly what the “visual-context accuracy” criterion (25% weight) needs. The 3b model handles interactive steps (click/type/search) fine but struggles with comprehension-heavy tasks; routing sends those to 7b.

5.3 End-to-end latency

Configuration	Per-step latency
qwen2.5v1:7b (single model)	~19.4 s (incl. ~0.9 s on-device vision + OCR gate)
Routed 3b/7b	~12.7 s avg ; interactive steps ~7.5–8.0 s

Per-step waterfall: *capture (about 82 ms) → serialise (0.5 ms) → vision (300 ms) → sanitise (92 ms) → zero-leak OCR gate (600 ms) → upload+VLM (8–19 s) → execute (1.2 s).*

5.4 Client resource utilization

Metric	Value
Model weights bundled	2.2 MB (BlazeFace short-range ~0.23 MB + Tesseract English ~0.1 MB)
Total on-device assets	21.4 MB (models + WASM runtime glue for MediaPipe + Tesseract)
Off-device fetches	0 MB (no CDN, no external API calls)
On-device vision per step	~0.3 s
Zero-leak OCR gate per step	~0.6 s
Total client-side work per step	~0.9 s

6 Feasibility and Honest Limitations

What works today

- **Runs on a 4 GB GPU laptop.** The full agent — extension + server + VLM + on-device vision — operates on an RTX 3050 with 4 GB VRAM. The routed 3b/7b design is what makes this possible: interactive steps use the small fast model; comprehension-heavy steps use the larger one.
- **Everything local by default.** Ollama + FastAPI bind to 127.0.0.1. Zero cloud dependency, zero API-token cost, works offline.
- **On-device vision is genuinely on-device.** 21.4 MB of bundled weights and WASM runtimes, never fetched from a CDN, Chrome-offline-safe.
- **Extension builds cleanly.** WXT produces a ready-to-load unpacked Chrome extension; typecheck passes; server runs with a single `uvicorn` command.
- **Benchmarks are reproducible.** The Playwright harness + metric runners produce JSON; the aggregation script merges them into a scoring-aligned dashboard.

Known gaps (stated honestly)

- **Latency target:** < 5 s/step is **not yet met** on the 4 GB dev laptop (measured 12.7–19.4 s/step). The scaling story: VRAM ≥ model size (judge-class GPU) makes it reachable; routed mode is the low-memory fallback. This gap is stated openly, not hidden.
- **Vision host is Chrome-only.** The offscreen-document architecture works in Chrome MV3 but not Firefox MV2 (no offscreen documents). The Firefox pass requires moving the vision host to the sidebar — planned, not yet done.
- **OCR on heavy media pages** (YouTube thumbnails, image-heavy news) is the slowest on-device path. Region caps and a sampled gate bound this, but it is the worst case.
- **Packaged build for the judging VM** (Chrome extension zip) is still to do.

7 Impact and Benefits

7.1 Privacy (primary impact)

- **Raw screenshots never leave the machine.** PII is redacted on-device before any payload is sent — shrinking the leak surface from *every screenshot of every form, every field, every face* to *redacted pixels + a numbered DOM*.
- **No cloud, no per-token cost, no data brokers.** A user can automate banking, book-keeping, and travel tasks locally — filling forms, reading dashboards, searching — while sensitive values ([EMAIL_1], [AADHAAR_1], [PAN_1], [SECRET]) never travel over the network.
- **Five independent defense layers:** Tier A (password box) + Tier B (regex PII tokens) + Tier C (face blur + image OCR) + zero-leak DOM scan + zero-leak image OCR. A failure at any layer blocks the upload. This is defense in depth, not a single heuristic.

7.2 Accessibility and productivity

A lightweight, ownable browser agent: automation of repetitive web work (form-filling, data-entry, flow regression) for individuals and organizations that cannot or should not ship their data to a commercial agent service. A local server means no API-token costs, no rate limits, and offline operation.

7.3 Why this matters for the judges

- **Metric-ready and reproducible:** PII F1 0.979, redaction precision 0.959, 3/3 task success, zero-leak PASS — every number regenerates from the automated harness. Judges can verify.
- **Demo-provable:** during a run, the browser’s Network tab shows only sanitised payloads leaving; the side panel shows live capture, VLM “thinking” tokens, and the redaction log side-by-side. No hand-waving.
- **Resource story (20% of score):** about 2.2 MB of actual model weights, 21.4 MB total bundled assets, on-device cost about 0.9 s/step — a strong “client resource utilization” case.
- **Feasible under constraint:** proven on a 4 GB GPU laptop. The architecture (routing + bundled on-device vision) scales down to modest hardware; the sub-5 s latency target is scoped to judge-class GPUs with adequate VRAM.

8 Build Chronology

The prototype was built in phases, each producing a testable increment:

Phase 0 — Scaffold. WXT extension skeleton (Chrome MV3), side-panel UI with task input + activity log, background orchestrator driving the capture→act loop, content-script executor (click/type/press/scroll/navigate/wait/extract/done) + DOM serializer (shadow-root aware, bbox × dpr), and the frozen v1 action protocol shared as TypeScript types on the client and Pydantic models on the server.

Phase 1a — Privacy gate. The core sanitizer and PII detection rules: Tier A/B regex redaction, field-label flagging (exact and substring matching), prose PII sweep, and stable-token generation so the VLM keeps continuity across steps.

Phase 1b — Zero-leak. The zero-leak scanner: outbound DOM-JSON regex scan, later joined by OCR of the sanitised screenshot. Both fail-closed; the fail-closed-on-exception behaviour was added after discovering that a crashing OCR gate must not silently approve.

Phase 1c/1d — Server and prompts. FastAPI app with `/health`, `/warm`, `/act`, and the VLM connection (initially `llava:7b`, then `qwen2.5v1:7b` via Ollama). Prompt templates are kept in a separate file so prompt iteration never touches serving logic. The `/rethink` endpoint was added later with structural action vetoing.

Phase 2 — On-device vision (major milestone). MediaPipe BlazeFace face blur (Tier C) + region-restricted Tesseract OCR, entirely on-device in an offscreen document with bundled weights. The taint policy (whole-region redaction on any OCR PII hit) was designed here. The vision-host architecture — the offscreen document as the only DOM context available to run MediaPipe/Tesseract in MV3 — was the hardest integration challenge.

Phase 3 — Benchmarks and measurement. GT-labelled test site (7 pages with `data-gt` attributes, including AI-generated faces and canvas PII), Playwright harness, mask-based precision/recall, zero-leak checks, latency waterfall, and the full VLM agent loop. Initial baseline: 0.881 F1. The rasterised-mask methodology replaced an earlier per-element approach that was undercounting.

Phase 4 — Meeting targets (in progress). Iterative improvements that brought F1 from 0.881 → **0.979**, task success to **3/3**, and step latency from 18.6 s → **12.7 s** (routed):

- VLM accuracy tuning for the prose “read & report” task that 3b was failing;
- executor hardening: auto-descend into real `<input>`s, verified-target memory, “forgot to type” guard;
- recovery: `/rethink` structural veto + untried-element escape hatch + within-run warnings;
- latency: `VLM_MAX_TOKENS`, compact prompt, image resize, `keep_alive`, **3b/7b dynamic routing**.

Still open before judging: sub-5 s latency on judge-class hardware, Firefox pass (vision host → sidebar), packaged Chrome zip for the VM, 3-minute demo video, pitch deck, and the architecture diagram.

9 Remaining Deliverables Checklist

Item	Status	Notes
Architecture diagram (SVG/Mermaid)	Not done	Can be generated from the loop diagram in §2.
3-minute demo video	Not done	(1) open the test site; (2) run the agent on a task; (3) show the Network tab proving sanitised-only payloads; (4) show face blur + OCR redaction.
Screenshots (3–4)	Not done	Side panel mid-task; sanitised image with SoM tags + redaction; metrics dashboard; server console.
Pitch deck	Not done	Suggested arc: Problem (agents leak data) → Our answer (sanitise on-device, ground by ID, local VLM) → Proof (0.979 / 0.959 / 3-of-3 / zero-leak + demo) → Feasibility (works on a 4 GB GPU) → Impact (privacy + productivity).
“What’s our edge” research claims	Not done	5–8 citations + 3 slide-ready claims comparing our approach to existing browser agents and cloud-based solutions.
Firefox pass	Not done	Vision host must move from the offscreen document to the sidebar (MV2 has no offscreen documents).
Packaged Chrome extension zip	Not done	<code>npm run zip</code> → <code>.output/chrome-mv3.zip</code> for the judging VM.
Judge-hardware latency note	Not done	State that sub-5 s is verified feasible on VRAM $\geq$ model size; routed 3b/7b is the low-memory fallback; own the dev-laptop gap explicitly.

10 Research Context (for “why our approach” slides)

10.1 The problem with existing browser agents

Most browser automation agents (commercial and open-source) operate in one of two modes, both problematic for privacy:

1. **Cloud-based agents:** screenshots are sent to a remote VLM (GPT-4V, Claude Vision, etc.). Every form fill, every bank page, every Aadhaar number is visible to the cloud provider. The user has no control over retention, training, or secondary use.
2. **Local-only agents without redaction:** the VLM runs locally, but the raw, unsanitised screenshot is sent to it. If the VLM is compromised, or the model itself memorises and leaks training data, the PII is exposed.

Our approach is **neither**. We redact *before* the image reaches the VLM — even a local one. This is a meaningful distinction: the VLM never sees raw PII, period.

10.2 What makes this technically novel

1. **Sanitise-before-send as a hard architectural constraint:** not a feature toggle, not “best effort” — the single choke-point design means there is literally no code path that sends unsanitised data.
2. **Stable PII tokens for VLM continuity:** replacing [EMAIL_1] with a stable token across steps balances privacy (the VLM never sees the email) with utility (the VLM can reason about “the email field” consistently).
3. **Set-of-Marks grounding by ID:** element-number grounding (rather than coordinate prediction) is more robust to DOM changes, does not require the VLM to learn coordinate systems, and works across page layouts.
4. **Two-layer zero-leak with fail-closed OCR:** re-scanning the *output pixels* (not just the input

- DOM) is a belt-and-suspenders approach borrowed from network security — DLP systems scan both structured data and rendered output.
5. **Routed small/big VLM for latency on modest hardware:** interactive steps (click/type/search) do not need a large comprehension model, while read/report tasks do; routing accordingly brings average latency down without sacrificing accuracy.

10.3 Comparison to the state of the art

Approach	Raw PII to VLM?	Cloud dependency?	Runs on 4 GB GPU?
Commercial agents (e.g. browser-use)	Yes (GPT-4V / Claude)	Yes	N/A
Open-source local agents (e.g. CogAgent)	Yes (raw screenshot)	No	Usually no (7b+ vision models)
This prototype	No (sanitised)	No (local Ollama)	Yes (routed 3b/7b)

The gap we fill: a privacy-preserving browser agent that is *both* local *and* redacts before the VLM sees anything — something no existing open-source agent does.